

Project 1

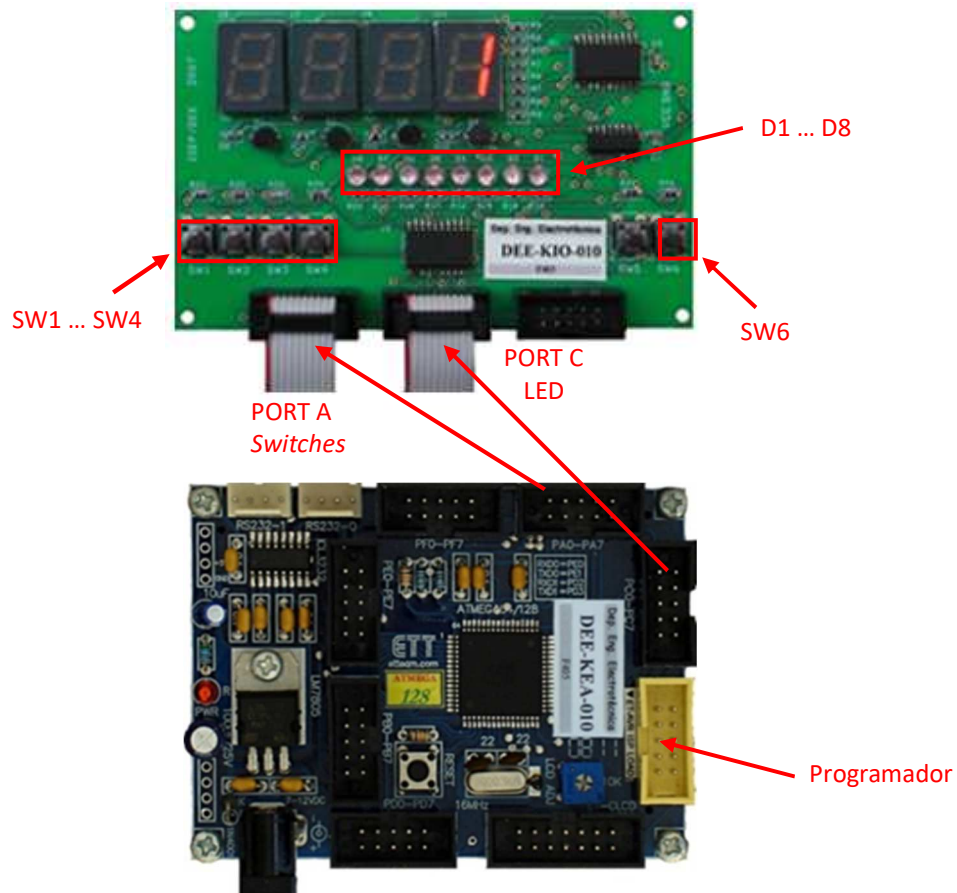
Objective: implementation of simple data input/output operations, usage of timers/counters and external interrupts.

Requirements: knowledge of the μC architecture, memory architecture and I/O ports, knowledge of Timers/Counters (TC0 in particular) and interrupts.

Operation 1: control the state of the LEDs D1...D8, using the switches SW. The LEDs state should obey the following table:

Active switch	LEDs ON
SW1	D1, D8
SW2	D2, D7
SW3	D3, D6
SW4	D4, D5
SW6	(All LEDs OFF)

Necessary hardware:



Switches port

Pin no.	Function	Pin no.	Function
1	SW1 (PA.0)	2	SW2 (PA.1)
3	SW3 (PA.2)	4	SW4 (PA.3)
5	SW5 (PA.4)	6	SW6 (PA.5)
7	MUX.0	8	MUX.1
9	Vcc	10	Ground

Note: according to the hardware of the I/O board, the default (OFF) state of the switches corresponds to the logical value 1, the active (ON) state corresponds to the logical value 0.

LEDs port

Pin no.	Function	Pin no.	Function
1	D1 (PC.0)	2	D2 (PC.1)
3	D3 (PC.2)	4	D4 (PC.3)
5	D5 (PC.4)	6	D6 (PC.5)
7	D7 (PC.6)	8	D8 (PC.7)
9	Vcc	10	Ground

Note: according to the hardware of the I/O board, to turn ON a LED the respective pin number should have the logical value 0, to turn OFF a LED use the logical value 1.

Software implementation

- Use **Assembly programming language**

Operation 2:

starting with all 8 LEDs (**D1 ... D8**) turned OFF, it is intended that, when pressing switch **SW1**, the LEDs are sequentially activated starting with LED **D1(D1->D8)**. The time between events must be **1 s**. When pressing **SW2**, it is intended that a sequence for deactivating the 8 LEDs initiates, starting with all LEDs turned ON. The LEDs must be sequentially deactivated in the reverse order, starting with LED **D8(D8->D1)**. For this case, the time between events must be **0,5 s**. Pressing **SW6** allows to stop the sequence of events (direct or reverse), that should restart when pressing either **SW1** or **SW2**.

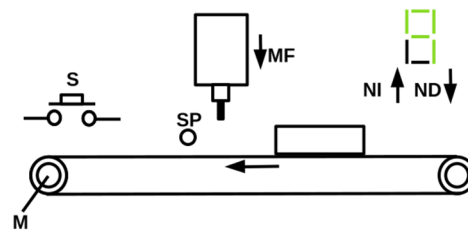
Software implementation

- Use **Assembly programming language**

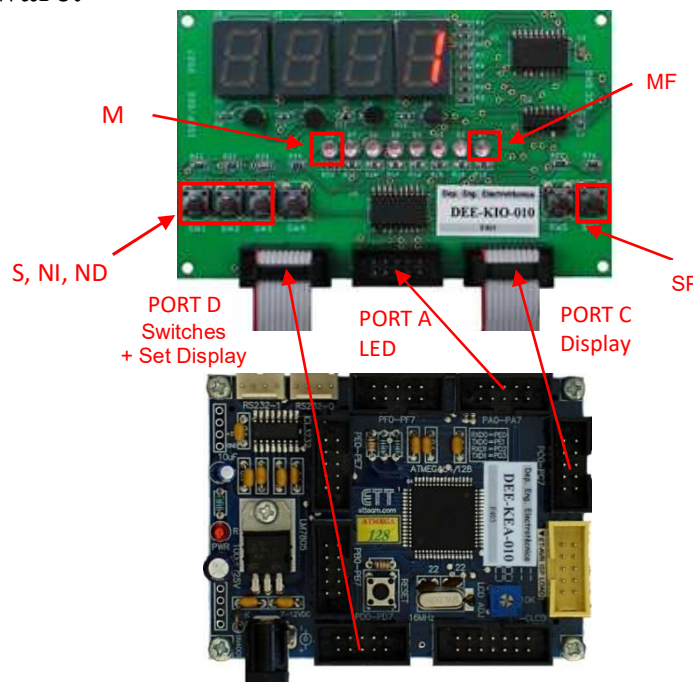
Operation 3:

The objective is to control a board drilling system. The system allows the selection of the number of boards for drilling, with default value of **9** boards. The system's operation starts when the **S** switch is pressed. The process starts by activating the motor **M** until the sensor **SP** detects the presence of a board. As soon as a board is detected, the motor **M** must be stopped and the motor **MF** activated, for **500 ms**, to start the drilling. Then, the motor **MF** must be deactivated. After a **500 ms** wait, the process is repeated the number of times corresponding to the number of boards selected. The number of boards is selected by the user and initiates with a default value of **9** boards. The user can change this value using the **NI** switch (to increment the value) and the **ND** switch (to decrement the value). This value must be defined after the process ends and before the **S** switch is pressed to start a new process. The minimum value is **1** board and the maximum value is **9** boards. The display on the right must show the number of boards that did not undergo the drilling process and must be updated during this process.

To determine the value for the **S**, **NI** and **ND** switches, two readings, separated by a **2ms** interval, should be obtained for each switch, and, only when the same logical value is obtained in both readings, is the reading considered valid and the respective value selected.



Necessary hardware:



Switches port

Pin no.	Function	Pin no.	Function
1	S (PD.0)	2	NI (PD.1)
3	ND (PD.2)	4	SW4 (PD.3)
5	SW5 (PD.4)	6	SP (PD.5)
7	MUX.0 (PD.6)	8	MUX.1 (PD.7)
9	Vcc	10	Ground

Note: PD.0 .. PD.5 should be programmed as data input to obtain the information from the switches. PD.6 e PD.7 should be programmed as data output and both pins should be updated with the logical value **1** to select the display placed on the right side.

Displays port

Pin no.	Function	Pin no.	Function
1	Seg a (PC.0)	2	Seg b (PC.1)
3	Seg c (PC.2)	4	Seg d (PC.3)
5	Seg e (PC.4)	6	Seg f (PC.5)
7	Seg g (PC.6)	8	DP (PC.7)
9	Vcc	10	Ground

Note: to turn ON one segment of the seven segment display, the respective pin number should have the logical value 0, to turn OFF one segment use the logical value 1.

Segment table

Digit	DP	Seg g	Seg f	Seg e	Seg d	Seg c	Seg b	Seg a	PORT C
0	1	1	0	0	0	0	0	0	0xC0
1	1	1	1	1	1	0	0	1	0xF9
2	1	0	1	0	0	1	0	0	0xA4
3	1	0	1	1	0	0	0	0	0xB0
4	1	0	0	1	1	0	0	1	0x99
5	1	0	0	1	0	0	1	0	0x92
6	1	0	0	0	0	0	1	0	0x82
7	1	1	1	1	1	0	0	0	0xF8
8	1	0	0	0	0	0	0	0	0x80
9	1	0	0	1	0	0	0	0	0x90

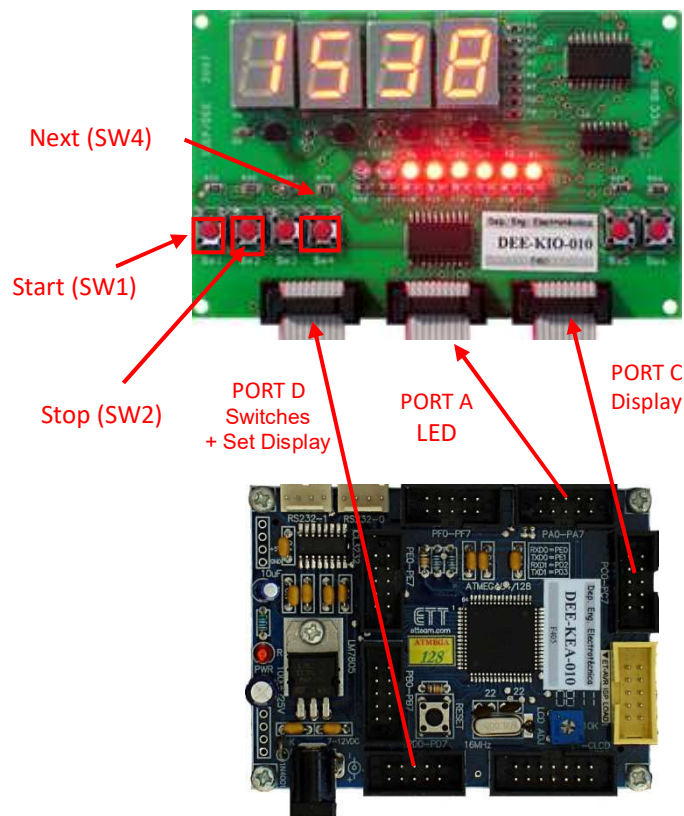
Software implementation

- Use Assembly programming language

Operation 4:

using one of the 7 segment displays, the objective is to implement a number roulette, that will display, sequentially, 6 digits. Every time the switch **Start** is activated, the display must show the digit “0” until the digit “5” at a **500 ms (2 HZ)** rate. When the digit “5” is displayed, the display must restart with the digit “0”. Activating the **Stop** switch, during the roulette operation, must interrupt the sequence and, the last digit displayed, must be shown blinking at a frequency of **1Hz**. After **3 seconds** the roulette finishes and the display must show the selected digit without blinking. In the initial state, the display must be turned OFF.

Necessary hardware:



Suggestion: use the TC0 in mode 2 (“Clear Timer on Compare”) to generate a time base of 2 ms.

Software implementation

- Use Assembly programming language

Operation 5:

using the last operation and the four 7 segment displays, the objective is to implement the Mastermind game. At the beginning of the game, with all 4 displays OFF, the first player must press the **Start** switch to choose the sequence of 4 numbers that the second player will try and guess. To do this, after the **Start** switch is pressed, the roulette operation must start on the left display. When the intended digit appears, the first player must press the **Next** switch to select the first digit and start the roulette operation in the next display. Every digit selected must remain visible for the rest of the sequence selection. When the first player is selecting the last digit (display on the right), it must end the 4 digit selection by pressing the **Stop** switch. The 4 digit sequence must remain visible, without blinking, for **3 s** and, then, all displays must be turned OFF.

The second player can now start guessing the correct sequence of digits (as selected by the first player). The second player has **8** attempts to do so. Therefore, LED **D1** must be turned ON to signal the first attempt and the second player must press the **Start** switch to initiate the selection of his first 4 digit sequence, in the same way as the first player. After pressing the **Stop** switch, it must be shown, in the corresponding displays the digits that the second player guessed right, for a time period of **3 s**. If all digits are wrong, then all four 7 segment displays must be OFF. If all digits are right, then, the second player won and the sequence of 4 digits must be shown, blinking, **3** times at a **1 Hz** rate. If the second player did not win, then the LED **D2** must be turned ON and the second attempt can start. If, after the 8 attempts, the second player is unable to find the correct sequence, he loses the game.

Software implementation

- Use Assembly programming language